

What CI covers — and why nothing says “this change hits NDF2W”

Business logic cannot be unit-tested effectively. Tests and CI cannot say whether a change affects NDF4W underwriting or NDF2W. Validation is applied only after the service is already serving traffic. “Still running” is treated as “healthy”. And the role of a nightly end-to-end run in proving the contract.

1,457 TEST FILES — 4 OF WHICH RUN A CAMUNDA PROCESS	248,685 NDF2W APPLICATIONS REACHABLE BY A “NEW GENERATION” EDIT	38,198 MODEL-PARSE WARNINGS EMITTED IN PRODUCTION, PER WEEK	6 of 9 TEST LAYERS THAT DO NOT EXIST — INCLUDING BOTH THAT RUN IN MILLISECONDS
---	---	---	--

Seven claims, graded against the build and against production

Business logic cannot be unit-tested effectively	● TRUE	And for a <i>sharper</i> reason than in LORA. 776 of 1,457 files are pure Mockito, and Mockito can reach Java. But Bravo’s routing logic is not Java — it is <code>conditionExpression</code> strings in 53 BPMN files, a six-column DB selector, and a per-application jsonb matrix. Those three are data, and no unit test executes any of them.
CI cannot distinguish an NDF4W-underwriting change from an NDF2W one	● TRUE, AND WORSE	Product identity is expressed in five places at once . And <code>ndf2w.bpmn</code> bridges into the <i>unified</i> underwriting sub-process, so a change to unified underwriting reaches NDF2W — 248,685 applications over 90 days — with nothing in the build saying so.
Validation is applied only after the service is serving traffic	● CONFIRMED	All 53 BPMN and 3 DMN files auto-deploy and parse at Spring Boot startup. The engine emitted 38,198 ENGINE-09004 warnings in production in 7 days , naming real modelling defects in real files. There is no parse or lint gate before that.
“Still running” is treated as “healthy”	● CONFIRMED, IN BRAVO’S EYES	“No 5xx” is treated as healthy. Every success-rate monitor filters <code>http.status_code:5*</code> . A 404 storm of 748,686 a week across 69% of surveyor sessions is, by that definition, a healthy service.
A nightly end-to-end run proves the contract	● NO SUCH RUN	The org has 8 Synthetics tests, all DNS and SSL checks, and zero CI pipeline events for any repository. LORA’s nightly partner-E2E is mostly red — but it exists, asserts a terminal condition, and can therefore be fixed.
The 1,457-file suite is a safety net for the orchestration	● NO	Four test files deploy and run a Camunda process, one against a production BPMN. There is no test of retry exhaustion, incident creation, escalation across a <code>callActivity</code> , or the forced-termination path.
Coverage is gated	● NO GATE EXISTS	<code>pom.xml</code> configures JaCoCo <code>report</code> and no <code>check</code> ; there is no threshold. The <code>makefile</code> passes <code>-Dspring-boot.run.profiles</code> , which surefire ignores — the tests are not running under the profile the build claims.

Bravo’s suite tests the Java it can reach and does not touch the XML, the configuration tables or the engine — which is where the loan’s behaviour is decided.

The result is a build that can be green for a change that alters the routing of 94% of production volume.

A STRUCTURAL DIFFERENCE IN BRAVO’S FAVOUR

LORA has 608 test files across 13 repositories with **four repos at zero** and `lora-process-sdk` carrying no `.github/workflows/` at all. Bravo has one repository, one build, and everything in it runs. Bravo’s problem is coverage of the right thing; LORA’s is that some repositories are not covered at all. **Bravo’s is the better problem to have.**

THE TELL

`camunda-bpm-assert` , `camunda-bpm-process-test-coverage` and `camunda-bpm-mockito` are all in `pom.xml` and all effectively unused. **Somebody knew this was the right way to test a BPMN system, added the dependencies, and the practice never took hold** — the same shape as LORA’s `lora-super-test` , which has 150 scenario tests and no CI runner.

Sections 1–6 diagnose. §7 is the architecture — nine cost-ordered layers L0–L8, a regression suite T1–T12 and a negative suite N0–N12, with a routing rule that says which layer should have caught what.

1,457 test files, 4 of which run a process

SURFACE	COUNT
Test files, <code>src/test/java</code>	1,457
Pure Mockito unit tests	776
Tests that deploy and execute a Camunda process	4
H2 “parse tests” asserting BPMN structure	a handful
End-to-end walks, application start to go-live	0
Tests of retry exhaustion, incident creation, cross- <code>callActivity</code> escalation, forced termination	0
Coverage threshold	none
One of the four Camunda-executing tests runs against a production BPMN, <code>unified-anti-fraud-engine.bpmn</code> . JaCoCo is configured for <code>report</code> only – no <code>check</code> goal, no floor.	

0.27%

OF TEST FILES EXERCISE THE ORCHESTRATION ENGINE – AND THE ENGINE IS WHAT DECIDES WHAT HAPPENS TO A LOAN

The suite is large, well-maintained and conventional. **Nothing in it is bad.** The problem is what it is pointed at.

WHERE THAT LEAVES THE SAFETY NET

1,453 of 1,457 files test Java that surrounds the process. The four that start a process do not cover the legacy-to-unified bridge, the retry policies, or the degrade decisions – and nothing else does either.

Where the business logic actually is — and why Mockito cannot reach it

In LORA this complaint was graded *overstated*, because LORA’s rules live in Go preconditions that `PreConditionIsMet(nil, data)` can execute directly — 97 test files already do. In Bravo the complaint holds, and the reason is that a large part of the decision surface is **not in Java at all**.

WHERE A ROUTING DECISION LIVES	FORM	CAN A UNIT TEST EXECUTE IT?
BPMN gateway conditions – 137 exclusive gateways in <code>ndf4w.bpmn</code> alone	JUEL strings inside XML	No – only by deploying the process and running it. 4 test files do this
Which activities run for this application	<code>WorkflowProductConfig</code> → <code>WorkflowMasterConfig</code> → per-application jsonb matrix	No – it is rows in PostgreSQL, shipped as Flyway data migrations
The skip gate itself	<code>BaseActivity.execute()</code> reads the matrix and runs or no-ops	Yes in principle – the <i>matrix content</i> is still data
Which BPMN process starts for a product	YAML map <code>productId</code> → key, <code>application.yaml:1364</code> , with a Java override for RO	Partly – and the map contains four keys that are not deployed processes
Approval chain rules	<code>BaseUnderwritingApprovalServiceImpl</code> , 4,604 lines, behind feature flags	Yes – genuinely Java and genuinely testable
Terminal vs transient error, and the degrade decision	50 <code>customErrorHandle</code> implementations invoked on the last retry	Yes for the handler; no for “does the engine actually reach the last attempt”

Bravo’s Java is testable and largely tested. Bravo’s **decisions** are in XML, YAML and database rows — and none of those is executed by any test in the build.

THE EXPENSIVE CONSEQUENCE

The `customErrorHandle` framework contains at least one **credit-policy** decision: bypassing anti-fraud after three failed attempts keeps the pipeline moving. Whether that path is ever reached depends on `failedJobRetryTimeCycle` — a string attribute on a BPMN element, **186 declarations, 30 distinct values, including 5 PT4M cycles with no repeat count that do not do what their author intended**. No test asserts that any activity’s retry policy is what the modeller meant, and the one that degrades anti-fraud is the one where being wrong is a credit-risk event rather than an outage.

Product identity is in five places

MECHANISM	WHERE
Numeric literals	<code>Application.isNDF4W() { return productId == 1L; }</code> – 18 such predicates
YAML process-key map	<code>application.yaml:1364</code> , plus an RO override in <code>ApplicationServiceImpl.java:752-762</code>
Six-column DB selector	<code>WorkflowProductConfig</code> → <code>WorkflowMasterConfig</code> → per-application jsonb matrix
Java factories	31 of them
Gateway string expressions	inside all 53 BPMN files

A change to any one of these can alter the behaviour of one product, several, or all. Adding DF2W Sharia (product 15) touched **all five**, behind per-product feature flags.

NOTHING MAPS A DIFF TO ITS PRODUCTS

There is no artefact in the build — no test name, no tag, no coverage report dimension — that maps a diff to the set of products it affects. **A reviewer’s only tool is knowing all five mechanisms.**

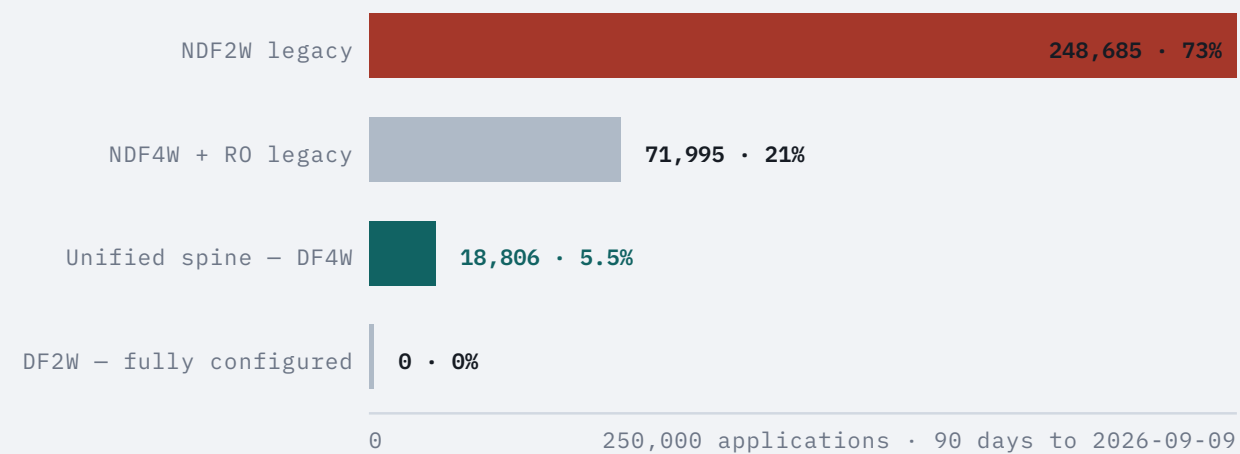
LORA’S ANSWER IS STRUCTURAL, NOT TESTED

Product families have separate document schemas, separate workers and separate task queues, so a change to `.../ndf2w` cannot reach NDF4W *by construction*. LORA still cannot tell you *which* product a shared-planner change affects — that is its own True verdict — but it has fewer ways to get it wrong.

The legacy-to-unified bridge

ndf2w.bpmn calls into the unified underwriting sub-process. And no callActivity in the codebase sets camunda:calledElementBinding, so every child resolves to the latest deployed version at call time.

PUT THOSE TWO FACTS TOGETHER WITH THE PRODUCTION VOLUME SPLIT



A change to unified-underwriting-*.bpmn — the “new”, 5.5%-of-volume generation — is picked up by in-flight NDF2W loans at their next call activity.

That is 73% of BFI’s origination volume, reached through a file whose name says it belongs to the other generation, with no migration plan, no version pin and no test.

NOTHING COVERS THIS PATH

The four Camunda-executing test files do not cover it. Neither does anything else. This is the most consequential untested edge in either platform, and it is invisible in the diff.

Validation happens in production, 38,198 times a week

The LORA complaint was that validation is “only JSON Schema, applied after the worker is already serving traffic”. Bravo has no schema registry at all, so the equivalent question is: *when is a process model checked? At pod startup, in production.* Spring Boot auto-deploys every BPMN and DMN resource on the classpath — there is no `deployment-resource-pattern` narrowing it — and the engine parses and versions each changed definition as it boots. **That parse is the validation.**

A REPRESENTATIVE LINE, CAPTURED LIVE

```
status: warn
version: v2.93.53
message: ENGINE-09004 Warnings during parsing:
  * Exclusive Gateway 'Gateway_X_KYC_Verified' has
    outgoing sequence flow 'Flow_Condition_KYC_Verified'
    without condition which is not the default flow.
    We assume it to be the default flow, but it is bad
    modeling practice, better set the default flow in
    your gateway.
  | resource unified-kyc-check.bpmn
@activityName: KYC Check
@processDefinitionId: 544e3ea0-ab85-11f1-ba59-16deca89b476
```

Read that carefully. The engine is telling the team, *in production*, that a gateway on the **KYC path** has an unconditional outgoing flow that Camunda is *guessing* is the default. That is a routing decision being resolved by an engine assumption rather than by the model. The warning names the file. It has been repeating for at least two deployed versions.

The honest comparison. LORA’s `docFieldCheck` panics the *worker at startup* if a schema path is missing — which is late, but is at least fatal and per-deployment. Bravo’s equivalent check emits a warning, assumes a default, and serves traffic.

DEPLOYED VERSION		ENGINE-09004	WARNINGS
v2.92.45			24,513
v2.93.53			13,685
Total, 7 days			38,198
Measured on <code>env:prod service:prod-ms-bpm</code> , 7 days to 2026-09-10.			

1 · VALIDATED AFTER DEPLOYMENT, NOT BEFORE

Every one of these warnings is available at **build time** — `camunda-bpm-assert` and a parse test would surface them in seconds — and none is a build failure today.

2 · THE COUNT IS A BOOT-FREQUENCY ARTEFACT

...and that is also informative. 38,198 in a week across two versions means **pods are restarting often** and re-parsing all 53 files each time. The *distinct* warning set is small; the volume tells you about pod churn.

3 · NOTHING CONSUMES THEM

No monitor matches `ENGINE-09004` . The warnings land in the same stream as the 525,181 error lines a week and are invisible.

“Not 5xx” = healthy

BRAVO DOES NOT HAVE LORA’S FAILURE CLASS

About half of LORA’s loans never reach a terminal status while producing zero errors and zero failed attempts — every signal reads healthy. Bravo’s BPMN checkpoints **fail fast into an incident and park**, so a wedged Bravo loan becomes a visible ticket rather than an invisible running workflow. That is a real advantage, and it is why Bravo’s stuck-application rate is *measurable* at ≈0.44% while LORA’s silent half went unnoticed for months.

BRAVO’S VERSION OF THE SAME MISTAKE IS ONE LAYER UP

```
(hits{service:prod-ms-bpm}
- hits{service:prod-ms-bpm, http.status_code:5*})
/ hits{service:prod-ms-bpm} < 99
```

Every health signal Bravo owns is defined as the **absence of a 5xx**.

Bravo’s release gate is it deployed and it is not returning 500.

That is the same category of statement as **the workflow is still Running**, and it is wrong for the same reason: it defines health as the absence of the one failure mode the system does not have.

REALITY	WHAT THE HEALTH MODEL REPORTS
748,686 HTTP 404s a week on the Surveyor Platform – 266,767 from one endpoint hitting 54,350 of 79,090 sessions	healthy
8,533 401 Unauthorized from the IAM permission endpoint, ~1,200/day, 1.07 errors per trace – a broad auth defect	healthy
20,802 outbound errors a week, none of them 5xx	healthy
~4,750 activity-level engine failures a day, One Obligor Check at 5,583 in 48 hours	healthy – nothing reads @activityName
Eight monitors permanently in Alert; six that cannot fire	–

The nightly E2E that does not exist

The role a nightly run plays for LORA is to prove the Digital Partnership contract: 11 partner journeys through DP → LORA, with a terminal assertion — the agreement number comes back on DP tracking status. LORA’s own testing.md grades that run harshly: **59 of 83 web cases have never passed in 31 nightly runs**, and the API run fails the same 199 cases every night. But the run *exists*, asserts, and is fixable.

BRAVO HAS NO EQUIVALENT, AND THIS WAS CHECKED THREE WAYS

SURFACE	WHAT IS THERE
Datadog Synthetics	8 tests in the entire org: 4 DNS checks (bfi.co.id, bfidigital.id, the two Bravo microservice domains) and 4 SSL certificate checks (microservices.prod.bravo.bfi.co.id, the Sharia equivalent, e-self.bfi.co.id, sso.bfi.co.id). Zero browser tests. Zero multi-step API tests. Nothing that creates an application.
Datadog CI Visibility	Zero pipeline events for any repository in 30 days – Bravo or LORA. No build, test or deploy telemetry reaches this org at all.
Repository	4 test files execute a Camunda process; no start-to-go-live walk exists

So the continuous evidence that a Bravo loan can still be originated end to end is: **the production ticket queue.**

Surveyor Platform - Release reject at 315 tickets in August, growing 4.8× since January, is the regression detector.

ONE CAVEAT IN BRAVO’S FAVOUR

Bravo’s volume is its own smoke test: **76,446 applications in August** means a total break in the main path is visible within minutes through the ticket queue and the latency monitors. **That works for outages.** It does not work for the failure Bravo actually has — a slow, product-specific, 4xx-shaped degradation that grows 82% over eight months while volume falls.

Sections 1–6 are a diagnosis. This is the architecture.

The diagnosis on its own has a failure mode: **nine recommendations with no ladder to hang them on become nine tickets that each get argued separately**. LORA’s *lora-super-test* does have a ladder — five `_`-prefixed layers (`_whitebox` , `_canary` , `_contract` , `_widgets` , `_utility`), a four-row cost budget, and a Zephyr tag per case — and its own account of *why*: full-SIT E2E “*tested their uptime, not our code*”. **Bravo has no equivalent structure at all**. What follows is derived from Bravo’s own decision surface rather than copied from LORA’s.

DESIGN RULE 1 – FROM LORA, STILL TRUE HERE

Test only what you control; own the answers at the boundary. LORA’s boundary is one gateway envelope. Bravo’s is different, and it sits somewhere better and somewhere worse.

DESIGN RULE 2 – THE WHOLE POINT FOR BRAVO

A change must fail in the cheapest layer that can see it. LORA states this as a rule of thumb. For Bravo it is load-bearing, because Bravo’s *expensive* layer — a Camunda process test — is the only layer that exists for orchestration today, and **it exists four times**.

WHERE BRAVO’S TEST BOUNDARY ACTUALLY IS

	LORA	BRAVO
Upstream call shape	one envelope, <code>POST /proxy/inner/request-v1</code> , naming its schema	113 typed <code>@FeignClient</code> interfaces , one per upstream
Interception point	one mock matching on <code>api_data_schema</code> answers as ~300 upstreams	113 stub sets — wider , but each is a typed Java interface
Stub drift caught by	<code>check:stubs</code> against the live LSS schema registry	the compiler . Bravo has no schema registry — but a stub built from the Feign DTO <i>cannot</i> drift without failing the build
Fail-on-demand today	per-run — a spec steers one answer with a 2-line delta	not possible . <code>bravo-mock-service</code> (Mockoon, <code>*.mock.bravo.bfi.co.id</code>) is static and shared — the same weakness LORA names when explaining why it built its own

The finding is the last two rows, and they run in **opposite** directions.

Bravo’s boundary needs no schema registry to stay honest, because 113 typed interfaces are checked by `javac` where LORA’s 300 JSON envelopes need a bespoke script. **That is a real, unearned advantage**. But Bravo has **no way to make an upstream fail on demand** — and the negative suite is the list of what that costs.

The nine layers, ordered by cost

Each row states what **only** that layer can catch — a layer that catches nothing a cheaper one sees does not belong on the ladder.

LAYER	WHAT IT OWNS	WHAT ONLY IT CAN CATCH	COST	TODAY
L0 · model lint	All 53 BPMN + 3 DMN parsed at build, no Spring context	ENGINE-09004 (38,198/wk in production); the 5 bare PT4M retry cycles among 186 declarations; the 4 YAML process keys that are not deployed processes; the 56 callActivity with no calledElementBinding; orphans among 70 error definitions and 190 escalations	seconds	none
L1 · config-table validation	The six-column selector, the productId → key YAML map, the per-application jsonb matrix, 1,350 Flyway migrations	Every productId resolves to a deployed process; every activity named in a jsonb matrix exists among the 274 distinct bean names; no WorkflowMasterConfig row points at nothing	seconds	none
L2 · Java unit	The 776 Mockito tests that exist. BaseUnderwritingApprovalServiceImpl (4,604 lines), the 50 customErrorHandle bodies	Ordinary Java defects. This layer is fine — it is the one Bravo already has	ms	776 files
L3 · gateway-expression eval	Every conditionExpression string, evaluated through ExpressionManager against a variable map — no process started	Which branch a given variable set takes, across 137 exclusive gateways in ndf4w.bpmn alone and ≈1.3 conditions per service task. The layer that turns Bravo’s routing from data back into code	ms	none
L4 · delegate contract	Each of the 276 JavaDelegate classes against stubbed Feign clients	What an activity writes to process variables; which customErrorHandle fires on last attempt — including the anti-fraud BYPASS credit decision	ms	partial, in L2
L5 · process fragment	One sub-process deployed and driven with camunda-bpm-assert — unified underwriting alone, unified KYC alone	Retry exhaustion, incident creation, escalation across one boundary, the forced-termination path	~1 s	4 files
L6 · product-matrix journey	One process test per generation × product, start → go-live, upstreams stubbed	“Does this change reach NDF2W?” — the question §3 says nothing answers. And the ndf2w.bpmn → unified-underwriting bridge	~10 s	none
L7 · console contract + widget	Snapshot the response shape of the 9+ FormTab sub-resources each console pane depends on; one live test per shared component	A console change that 404s a pane; the shape drift behind Cannot read properties of null (6,445/wk)	~2 s	none
L8 · production canary	A Synthetics multi-step API test that creates a real application; the per-console RUM error-per-view SLO	That origination works right now, on real infrastructure, with real upstreams	minutes	8 DNS/SSL

Six of the nine layers do not exist — and the two that would have caught the most both run in milliseconds.

L0 and L3 are the two Bravo has never built, and between them they cover every finding in §2: the XML, the YAML and the config rows that Mockito cannot reach are all reachable without the engine.

The routing rule: when something breaks, which layer should have caught it

SYMPTOM	CHEAPEST LAYER THAT CAN SEE IT	EXISTS
A gateway has no default flow	L0	no
A retry cycle does not repeat	L0	no
A new product has no deployed process	L1	no
A jsonb matrix names an activity that no longer exists	L1	no
An approval-ladder rule is wrong	L2	yes
A change reroutes one product and not another	L3	no
An activity writes the wrong variable	L4	partial
An upstream failure does not create an incident	L5	4 files
A unified edit changes an in-flight NDF2W path	L6	no
A console pane 404s	L7	no
Origination is broken in production	L8	DNS only

8 of 11

SYMPTOMS HAVE NO LAYER AT ALL – AND THE TWO IN BOLD CARRY THE LARGEST PRODUCTION EXPOSURE IN THIS DOCUMENT

THE ORDERING THIS IMPLIES IS DIFFERENT FROM THE RECOMMENDATION LIST – AND BETTER

L0 AND L1 SHOULD BE FIRST

Recommendation 2 is written as “fail the build on ENGINE-09004” – but the harness it needs is L0, and once L0 exists it **also** delivers T1, T3, T12 and the detection half of recommendation 1, in the same few days. **L0 and L1 are the cheapest work in this document and they close the largest number of findings.** They belong ahead of the item currently marked highest-value.

THE RULE OF THUMB, RESTATED FOR BRAVO

A form change fails in L7 in two seconds. A routing change fails in L3 in milliseconds. Only a real flow break should ever cost an L6 journey – which is why there are exactly **five** of those.

The regression suite, T1–T12

A regression test exists *because something happened*. Every case below is anchored to evidence in this pack, and the layer column says where it belongs — **not one of them needs a browser**.

#	REGRESSION	BECAUSE	LAYER
T1	<code>Gateway_X_KYC_Verified</code> has an explicit default flow	The engine is guessing it in production, on the KYC path, across two deployed versions	L0
T2	An NDF2W instance enters a pinned underwriting definition version	<code>ndf2w.bpmn</code> bridges into unified underwriting; 56 call activities unpinned; 248,685 applications / 90 d	L6
T3	Every <code>failedJobRetryTimeCycle</code> is a valid <code>Rn/PTn</code>	186 declarations, 30 distinct values, 5 bare PT4M that do not repeat	L0
T4	Anti-fraud degrades to <code>BYPASS</code> only on the last attempt	A credit-policy decision reached by an exception handler, verified by nothing	L4 + L5
T5	<code>feature-configuration</code> returns its documented status	266,767 404s/wk across 69% of surveyor sessions , on the busiest handler in the service	L7
T6	Every key in the <code>productId → process</code> map is a deployed process	The map contains four that are not	L1
T7	The approval ladder resolves after a role rename	<code>BLCS-4683</code> renamed NMH→GMB; <code>BLCS-4811</code> is reversing it	L1 + L2
T8	The IAM token is refreshed before <code>/permission/assigned</code>	8,533 401 s/wk at 1.07 per trace — a broad auth defect, not a retry loop	L4
T9	Adding a product touches all five discrimination mechanisms consistently	Product 15 (DF2W Sharia) touched all five, behind flags, untested	L1 + L3
T10	Position capture succeeds, or degrades explicitly	~20,700 geolocation failures/wk on a field-staff console	L7
T11	The <code>FormTab</code> v2 pane set matches its snapshot	9+ sub-resources; four are the top sources of the 404 storm	L7
T12	Every monitor's <code>service: tag</code> matches a registered service	Four Sharia monitors filter <code>prod-sharia-bpm</code> against <code>prod-sharia-bpm-sharia</code> and cannot fire	L0
Eight of the twelve land in L0, L1 or L7 — layers that run in seconds. Only T2 needs the expensive layer.			

The negative suite, N0–N12

#	FORCE THIS	ASSERT	LAYER
N0	Upstream returns 5xx	retries per <code>failedJobRetryTimeCycle</code> , then an incident is created and the loan parks	L5
N1	Upstream returns 401	the token is refreshed and the call retried – not counted as a terminal failure	L4
N2	Upstream returns 404	treated as “absent”, not as an error – the live repeat-order case, 5,070/wk	L4
N3	Upstream returns 400	a contract violation is terminal and named, not retried 186 times	L4
N4	Upstream never answers	the 300 s global <code>readTimeout</code> is not the effective timeout for a job-executor thread	L4
N5	Retries exhaust	the correct one of 50 <code>customErrorHandle</code> implementations fires	L4 + L5
N6	Anti-fraud fails 3×	degrade to BYPASS – and it is recorded as a credit decision , not a log line	L5
N7	Well-formed payload, wrong-typed field	rejected at the client, not written to the aggregate	L4
N8	The jsonb matrix disables a mandatory activity	the process refuses to start, rather than skipping a required check	L1 + L5
N9	A child sub-process throws	escalation reaches the parent through one of the 190 escalation paths	L5
N10	A loan is force-terminated mid-flight	status and <code>application_status_log</code> are consistent; no orphan job	L5
N11	The same application starts twice	idempotent – one process instance, <code>RetryLog</code> unambiguous	L4
N12	Two approvers decide concurrently	one wins; the ladder does not skip a tier	L2

Bravo cannot make an upstream fail at all.

LORA can make any upstream fail per run with a two-line delta. `bravo-mock-service` is static and shared, so “ask anti-fraud to reject” is not a thing a Bravo test can do. That is why 50 `customErrorHandle` implementations, 70 error definitions and 190 escalations are entirely unexercised.

AND BRAVO’S VERSION IS THE CHEAPER ONE TO BUILD

N0–N12 need one piece of infrastructure — the same piece LORA built: a **per-run stub layer in front of the 113 Feign clients**, where a test steers one answer and everything else comes from a shared baseline. Bravo’s stubs generate from *typed interfaces*, so there is **no schema registry to keep in sync** and **no `check:stubs` script to write**.

```
// the shape a Bravo negative case should have
stubs.baseline(NDF2W_SCORING_1);
stubs.antiFraud().failsWith(503).times(3);
// then assert the degrade, not the log line:
assertThat(application)
    .hasCreditDecision(BYPASS)
    .recordedBy("customErrorHandle");
```

400 cases must not mean 400 process runs

LORA’s arithmetic applies unchanged: 400 cases × a 2-minute walk is 13 hours, and a suite nobody can run in a sprint is a suite nobody runs. The same 400 cases distributed down the ladder:

LAYER BAND	CASES	EACH	WHEN	WALL CLOCK
L0 + L1 – lint and config	~60	~20 ms	every build	~2 s
L2 + L3 + L4 – Java, expressions, delegates	~1,100	~5 ms	every build	~15 s
L5 – process fragments	~60	~1 s	every build	~1 min
L6 – product-matrix journeys	5	~10 s	every merge	~1 min
L7 – console contracts	~40	~2 s	every merge	~1.5 min
L8 – canary	1	~3 min	hourly	–
Per-merge total				≈ 4 min

Five L6 journeys is the whole product matrix: NDF2W, NDF4W, RO, unified DF4W, DF2W.

Five tests would cover the paradigm’s happy path across 100% of production volume, against the zero that cover it today — and they are the expensive layer, deliberately kept to five, because L0–L4 catch everything that does not need the engine.

§7.6 · THREE RULES THAT KEEP IT FROM ROTTING

1 · NO HAND-WRITTEN EXPECTATION LISTS

LORA derives its 24-stage activity plan from real traces because hand-written lists “were proven wrong twice”. Bravo’s equivalent is the per-product activity set: **derive it from act_hi_actinst, never type it**. This depends on the job-executor spans in bravo-observability.md rec 8 — **without them, a derived Bravo expectation has no source**. That is the one dependency this document has on another.

2 · NO SELECTORS OR FIELD PATHS IN A TEST

Bravo’s version: **no productId == 1l literal in a test**. A test asks the config layer which product it is — so a test cannot become the sixth discrimination mechanism.

3 · NO STUB COPIED INTO A CASE

Name a baseline bundle, state only the delta. This is what makes N0–N12 two lines each instead of a fixture per case.

AND ONE RULE THAT IS BRAVO’S ALONE

A snapshot is never auto-recorded. LORA states the reason precisely — if the first passing run records the baseline, “that day’s form — bugs included — would silently become the truth”. For Bravo the trap is larger, because **L0 would otherwise record 38,198 parse warnings a week as the approved state of the models**.

Eleven actions — and where each lands on the ladder

1 Pin <code>calledElementBinding</code> , or write the one test that proves the bridge · S-M, HIGHEST VALUE 56 call activities, none version-pinned, and <code>ndf2w.bpmn</code> calls into unified underwriting. Either pin the binding so a unified edit cannot reach 73% of volume mid-flight, or add a process test that starts an NDF2W instance and asserts which underwriting version it enters. Today neither exists.	2 Fail the build on <code>ENGINE-09004</code> · S The warnings are already produced — simply in the wrong place. A parse test over all 53 files with <code>camunda-bpm-assert</code> , already in the pom, turns 38,198 production warnings a week into a red build. Start with the KYC gateway the engine is currently guessing about.
3 Add a 4xx clause to the release health gate · S “Not 5xx” is not health. At minimum, per-resource 404 and 401 rates on the four surveyor endpoints and the IAM permission call.	4 Build one end-to-end process test per generation · M Two tests: an NDF2W instance and a unified DF4W instance, each walked start to go-live with upstreams stubbed, asserting terminal status and the activity set executed. Two tests would cover the paradigm’s entire happy path, currently covered by none.
5 Test the retry and degrade policies · M Assert <code>failedJobRetryTimeCycle</code> is a valid <code>Rn/PTn</code> on every service task — the 5 bare <code>PT4M</code> cycles are a latent defect — and drive an activity to last-attempt to assert which <code>customErrorHandle</code> fires. The anti-fraud <code>BYPASS</code> is a credit-policy decision reached by an exception handler and nothing verifies when.	6 Make the product blast radius visible in the diff · M Tag every test with the products it covers and emit, per PR, the set of <code>productId</code> s reachable from the changed BPMN files, config rows and factories. A reviewer should not have to know five discrimination mechanisms to know whether a change reaches NDF2W.
7 Turn on a coverage gate, and correct the <code>makefile</code> · S JaCoCo <code>check</code> with a floor at the current level, ratcheting. Fix <code>-Dspring-boot.run.profiles</code> , which surefire ignores.	8 Add one Synthetics multi-step API test for the origination journey · S-M Not a replacement for a process test — a canary. Today the only continuous evidence that Bravo works is a support-ticket queue.
9 Send CI events to Datadog · S Zero pipeline events exist for either platform, so no one can answer “is the build getting slower, flakier, redder”. A configuration change that benefits both teams.	10 Build the per-run stub layer in front of the 113 Feign clients · M — UNBLOCKS N0-N12 Today no Bravo test can make an upstream fail, so 50 <code>customErrorHandle</code> implementations, 70 error definitions and 190 escalation paths are unexercised . <code>bravo-mock-service</code> (Mockoon) does not solve it — it is static and shared, precisely why LORA built its own. Bravo’s version is cheaper: stubs generate from typed Feign interfaces, so there is no schema registry to sync.
11 Derive expectations, never type them · S — A POLICY, NOT A TASK LORA derives its activity plan from real traces because hand-written lists were proven wrong twice. Bravo’s equivalent is the per-product activity set, derived from <code>act_hi_actinst</code> — which needs the job-executor spans in bravo-observability.md rec 8 first. Until then an L6 journey’s expected activity set has no trustworthy source , and that is the one dependency this document has on another.	

30, 60, 90 days

Eleven recommendations, phased against ~25 **person-days a month** of Squad S&U time — the allocation set by bravo-people.md recommendation 4. **The phases are the ladder, built cheapest-first** — L0 and L1 in month 1, L3 and L4 in month 2, L5 and L6 in month 3. That ordering is not a preference: *L6 is the only layer that can prove the `calledElementBinding` fix, and L0 is the only layer that can find all 56 call sites in seconds.*

SEQUENCING RULE

Investigate before touching anything that reaches in-flight loans. Recommendation 1 is the highest-value item here and the most dangerous to rush — unpinned `callActivity` binding affects **248,685 NDF2W applications per 90 days**. It is therefore read-only in month 1, a decision in month 2, and a change in month 3. Nothing else in this plan alters in-flight behaviour at all.

SHARED WITH OTHER DOCUMENTS

Recommendation 1 is also **bravo-delivery.md rec 6**; recommendation 3 is also **bravo-observability.md rec 2**. Both are phased identically in all documents — do them once.

Days 0–30

LOOK, AND START REPORTING

- 1 `calledElementBinding` investigation — read-only. Enumerate all 56 `callActivity` elements; map which children an NDF2W instance reaches; quantify the bridge exposure. *S&U · 5 d.* A written exposure note. **No code change this phase.**
- 9 Send CI pipeline events to Datadog. *Platform · 2 d.* Zero pipeline events becomes a build-health number, for both platforms.
- L0 Build L0 — model lint. All 53 BPMN + 3 DMN parsed at build with `camunda-bpm-assert`, already in the pom. *S&U · 4 d.* Delivers rec 2 plus **T1, T3, T12** and the *detection* half of rec 1. **38,198 production parse warnings a week become a red build that runs in seconds.**
- L1 Build L1 — config-table validation. *S&U · 3 d.* The **four YAML keys that are not deployed** processes fail the build. **T6, T7, T9** land here.

Days 31–60

PUT THE GATES IN

- 2 Fail the build on `ENGINE-09004`, and fix the KYC gateway whose default flow the engine is guessing. *S&U · 4 d.*
- 3 Add the **4xx clause to the release health gate**. *Platform/SRE · 3 d.* A release that 404s two-thirds of sessions fails the gate.
- 7 Coverage gate + `makefile` fix. *S&U · 2 d.*
- 8 Synthetics multi-step API test for origination. *QA · 3 d.*
- 1 `calledElementBinding` decision — pin, or accept in-flight child migration deliberately, with a migration plan. *S&U + EM · 2 d.*
- L3 Build L3 — gateway-expression evaluation. Every `conditionExpression` evaluated through `ExpressionManager`, no process started. *S&U · 5 d.* **Bravo's routing stops being data and becomes code** — and this plus L1 is rec 6.
- 10 Build the per-run stub layer in front of the 113

Days 61–90

PROVE THE JOURNEY, THEN MAKE THE CHANGE

- L6 Build L5 + L6 — the five product-matrix journeys. NDF2W, NDF4W, RO, unified DF4W, DF2W, each start to go-live on the phase-2 stub layer. Expectations **derived** from `act_hi_actinst`, never typed. *S&U + QA · 12 d.* The journey question is answered **for 100% of production volume** instead of 0%. **T2** lands here.
- 1 Implement the binding decision — pin all 56 call activities, with the phase-2 migration plan. *S&U · 10 d.*
- 5 Retry-policy validation, part 1. Assert every `failedJobRetryTimeCycle` is a valid `Rn/PTn`. *S&U · 4 d.*

Deferred

WITH TRIGGERS

- N The negative suite, N0–N12. Force each upstream failure mode and assert the degrade decision; **N6** — the anti-fraud `BYPASS` — is a credit-policy decision reached by an exception handler. Needs *both* the phase-2 stub layer and the phase-3 L5 harness; ~2 days per case once both exist. *Trigger: after the L6 journeys land.*
- L7 L7 — console contracts. Snapshot the 9+ `FormTab` sub-resource shapes; one live test per shared component. **T5, T10, T11**. The phase-2 4xx gate *detects* these; L7 is what *prevents* them. *Trigger: after the console inventory in bravo-delivery.md rec 7.*
- 6 Product blast radius in the diff. 8 days, and much cheaper once job-executor tracing and the flag registry exist. *Trigger: Q2 week 1.*

What changes when this is done

RECOMMENDATION	EXAMPLE WHEN DONE
<code>calledElementBinding</code> pinned	A change to unified underwriting cannot silently alter the path of 248,685 in-flight NDF2W applications. Migration becomes a decision instead of a side effect.
<code>ENGINE-09004</code> fails the build	The KYC gateway that Camunda is currently guessing the default flow for is fixed in a PR, not discovered in a log stream nobody reads.
4xx in the health gate	A release that starts 404-ing on two-thirds of surveyor sessions is caught at deploy, not by a rising ticket category.
Two end-to-end process tests	“Can a loan still get from submission to go-live?” has an automated answer for both generations, every build.
Retry and degrade tested	Nobody discovers by incident that anti-fraud has been bypassing after three failures, or that five retry cycles never repeat.
Product blast radius in the diff	A reviewer sees “this PR reaches NDF2W, NDF4W and R0” on the PR itself, and the five discrimination mechanisms stop being tribal knowledge.
CI events flowing	Build health becomes a number for both platforms instead of an impression.
L0 + L1 exist	A gateway with no default flow, a retry cycle that never repeats, a product with no deployed process, and a jsonb matrix naming a deleted activity all fail the build in seconds – instead of in a production log stream nobody reads.
L3 exists	“Which products does this diff reroute?” is answered by evaluating the gateway conditions, not by a reviewer who happens to know all five discrimination mechanisms.
The stub layer exists	A test can say “this run’s anti-fraud rejects”. 50 <code>customErrorHandle</code> implementations, 70 error definitions and 190 escalation paths stop being unexercised.
The ladder has a routing rule	A form change fails in L7 in two seconds, not in a 10-second journey three layers from the cause – and the per-merge suite still finishes in about four minutes .

Today the build can be green for a change that reroutes 94% of production volume. Two tests and one attribute close that.